

An Approach to Clean Architecture for Microservices using Python

Isha V P

Department of Computer Science and Engineering
R.V. College of Engineering
Karnataka, India
ishavp.cs19@rvce.edu.in

Vishalakshmi Prabhu H

Department of Computer Science and Engineering
R.V. College of Engineering
Karnataka, India
vishalaprabhu@rvce.edu.in

Abstract— Microservices architecture (MSA) is a popular approach to create maintainable and independent units of large systems. Recently many works have been carried out in migrating monolithic systems into MSA as well as using it in upcoming areas of development like Internet of Things (IoT). Most of the existing work emphasizes MSA for the system, little attention is paid to the architecture followed in the implementation of the underlying services. Clean architecture (CA) is a type of Domain Driven Design (DDD) approach that separates business logic so that it can be tested independently. Clean architecture is often used in Android systems. Introducing clean architecture in MSA can further ensure Separation of Concerns (SoC) and testability of business logic in the system. Through this paper an optimal solution to clean architecture in microservices is proposed. The system under consideration is a financial system that uses microservices based on Python and GraphQL.

Keywords— Business logic, Clean Architecture, Domain Driven Design, GraphQL, Microservices architecture (MSA)

I. INTRODUCTION

Microservices introduce an architectural pattern in which a system or an application is structured as a collection of services. The microservices approach amounts to loosely coupled, independent and business logic-oriented services. They enable organizations to rapidly develop systems in isolation by separate teams, even in different tech stacks. These services are reusable, maintainable, scalable and can be integrated when needed. It is highly important to have a good service architecture, business role definition and Application Program Interface (API) communication. Development team independence, simplicity in development, faster deployment, multiple tech stacks and domain segregation are some of the many advantages of microservice architecture over monolithic architecture.

One of the important choices to make while designing microservices is the architectural design of the services. In systems that place more significance in the business logic, DDD is a good option. DDD is a software approach where the system is developed by focusing on the business requirements. The core layer domain layer would hold all the business value and logic. Generally Model Driven Engineering (MDE) and Model Driven Architecture (MDA) are preferred for translating into DDD architecture. Out of the available options of DDD architectures, clean architecture is one of the sought-after choices. Clean architecture is a system architecture guideline that separates the system into layers that place the business logic or domain logic at the core.

This paper aims to propose a clean architecture-based solution for microservices written in python. The system in consideration was an existing financial Minimum Viability Product (MVP) that was being migrated to Microservice architecture. Despite microservices being modular and

independent in nature, introducing clean architecture further ensures Separation of Concerns within each service. The clean architecture model adapted in this system is in accordance with the Clean Architecture principles proposed by Robert C. Martin (Uncle Bob). Using clean architecture in microservices certainly helps with improving code maintainability, scalability and testability.[1]

II. RELATED WORKS

A. Microservices Architecture

Due to its many advantages, Microservice Architecture (MSA) is quickly gaining popularity in software engineering. The empirical research done on MSA as well as the new areas that have embraced MSA are discussed in the section that follows.

Zhiding et. al. [6] discusses a microservice extraction method from existing monolithic systems based on the information extracted from knowledge graphs. To identify the services in MSA, Louvain algorithm is proposed. Salah et. al. [7] shows the evolution of software designs from client-server architecture to Service Oriented Architecture (SOA) to microservices, to cope with increasing performance needs.

Muhammad et. al. [8] conducted a systematic mapping study to identify and analyse the concerns MSA poses to DevOps. In the light of continuous development of Internet of Things (IoT), monolithic architecture inhibits scalability. In Sun et. al. [9] a general MSA framework is proposed for IoT applications.

While most of the existing studies on MSA focus on architectural principles and application of architectural patterns, Shanshan et. al. [10] highlights the indispensable concern of the trade-off that microservices offer between quality attributes

(QA) like performance and scalability through a systematic literature review. It also highlights the relevance between MSA and significant QAs such as scalability, performance, availability, monitorability, security, and testability. Along similar lines, Alshuqayran et. al. [11] identifies the architectural challenges and QA related to MSA. A systematic mapping study was conducted to study microservices and their implementation.

MSA being one of the most sought-after architectural style, security concerns are an important concern to be answered. Abdelhakim et. al. [12] succeeds in identifying most relevant threats, proposing detection and mitigation mechanisms too. Finally, Aderaldo et. al. [13] proposes and discusses the use of an initial set of requirements that can be used as a benchmark for repeatable microservices research in the future.

A review of the MSA literature reveals that while many efforts have been made to incorporate MSA into a variety of emerging disciplines and empirical studies have been conducted, less effort has been made to choose the architecture for each of the microservices' individual services.

B. Clean Architecture

Systems that prioritize business logic more often employ clean architecture as a design guideline. Additionally, it is a good idea for systems that aim for maintainability, re-usability, and simplicity. Most mobile platforms or Android applications look for the flexibility and reliability offered by CA.

Software technology is developing quickly, which causes more issues with maintenance. A study was carried out by Nugroho et. al. [14] to confirm the positive association between codebase backend maintainability and CA. Layering in CAs effect on system performance was examined using metrics including Halstead's score, Cyclomatic Complexity, Weighted Method Count, Maintainability Index, and Kan's Defects. In their discussion of maintainability in agile teams, Arango et. al. [15] incorporated CA. By adhering to the SRP, OCP, LSP, ISP, and DIP principles (SOLID principles) of CA, maintainability was increased across each sprint in the scrum framework. Deduplication of the code was also noticed.

Strong business logic can be another justification for implementing CA in addition to maintainability. In Brown et. al. [16] a server side of an intelligent tutoring system that uses Situation Calculus as the core logic was refactored into CA. For the same Logtalk protocols and categories were used. Whitby adopted CA for both code re-usability and extensibility. The significance of CA in the development of client-side applications is emphasized by Bukovan et. al. [17]. They demonstrate the CA's robustness, upgradeability, testability, and maintainability.

Unlike systems that have strong business logic, in Aguilar et. al. [18] CA is used to build a system that needs communication between multiple data providers.

One of the sectors that uses CA the most is mobile platforms and Android applications. A MDE and MDA based method is suggested by Sanchez et. al. [19] to produce Android applications that make use of CA. This strategy made it possible to scale the system without compromising the business rules. To address the state management problem in the flutter development process, Boukhary et. al. [20] suggest a flutter architecture based on CA. The increased testability following the restructuring of the previous Rx Java application with CA is portrayed by Bui et. al. [21].

III. DOMAIN DRIVEN DESIGNS

Microservices is an architecture model wherein systems are created with loosely coupled services. They are easy to develop and maintain when aggregated into a complex system. They are bound to a specific context, configuration and dependency which result from the principles of DDD and DevOps.

DDD is an approach to achieve business goals through clear interfaces and functions. Hence microservices can achieve independence from other microservices by using DDD. A design in DDD includes the following layers:

- **User Interface Layer:** The system users interact with this layer to provide all their inputs and their interpretation is handled here too.
- **Application Layer:** This layer routes from one interface to another, including other system's interfaces. It also directs the domain objects to the Domain layer.
- **Domain Layer:** The domain rules and operational behaviour reside here. All the entities, which are combination of data and behaviour reside here.
- **Infrastructure Layer:** This layer facilitates communication between other layers.

Due to its simple communication, enhanced flexibility, and significant importance to the domain, DDD is extremely advantageous. Unfortunately, it is not appropriate for systems with high levels of sophistication. Layered architecture, hexagonal architecture, onion architecture, screaming architecture, and clean architecture are some of the well-known DDD techniques. [2]

- 1) **Layered Architecture:** Layered architecture is one of the most popular choices of architecture today. It works on the principle of Separation of Responsibility, which means an upper layer can access a lower layer but not the other way round.
- 2) **Hexagonal Architecture:** Hexagonal architecture portrays a practical model of Dependency Inversion wherein the domain layer becomes the central layer. It is also known as Ports and Adapters architecture, which denotes the interfaces and implementations respectively.
- 3) **Onion Architecture:** Onion architecture has multiple concentric layers that follow the Inversion of Control principle. The core layer is the domain layer here. This model separates the external dependencies of the domain entities as much as possible.
- 4) **Screaming Architecture:** As the name suggests Screaming Architecture must let any developer understand the main idea of the system in a glance, like the functionality and purpose of the system.
- 5) **Clean Architecture:** Clean architecture is yet another architecture used recently in Android applications vastly. This model also caters to the Separation of Concern principle primarily. It is explained in detail in the Section IV.[4]

IV. CLEAN ARCHITECTURE FOR MICROSERVICES

Clean architecture is a software design that follows the concept of clean code and SOLID principles. This design is a combination of best practice principles to emphasize the design over the domain logic and minimizing dependencies. It also includes decoupling the components of the system to reinforce isolation and maintainability. Business logic or domain logic is the part of the code that converts the real-world goals into software systems. Clean architecture divides software into layers to achieve Separation of Concerns like the other DDD architectures. It was proposed by Dr. Robert C Martin, famously called Uncle Bob. [3]

Clean Architecture following systems can be arranged into the four layers, entities, use cases, interface adapters, and frameworks and devices. These layers can be thought of as an arrangement of concentric circles architecturally, the same is depicted in Fig.1.

The innermost layers represent the entities while the outermost layers represent the practical implementations. Nothing from the inner layers must have access through the name of any outer layer's components. Similarly, no changes in outer layers must affect the inner layers. This means abstraction at its finest at the inner layers. These are some key points about architecture.

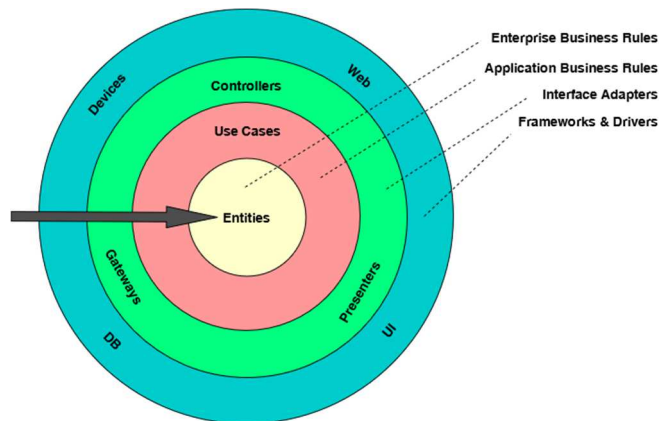


Fig. 1. Clean Architecture

1) **Entities:** Entities are the encapsulation of the larger business rules specifically the enterprise-wide critical domain rules. They can be a bunch of data structures or methods. They need to be reusable by many enterprise level applications. They are least likely to change in case of any operational change to a particular application. They are the highest level of abstraction. In python this layer mostly consists of classes, with methods for interaction. Entities in the same layer can interact with one another. In Python, one entity can communicate with another by using instances of the other class.

2) **Use Cases:** In use cases the system specific business rules which are the sole reason for the system to exist, reside. They encapsulate and give the implementation of all the use cases which work with real data in the system. They also depict the flow of data to and from the entities. They can instantiate the entities to use them. Any change in this layer should not affect the entities. Similarly, any change in frameworks must not affect this layer.

3) **Interface Adapters:** In interface adapters data is converted from the form that entities and use cases use to the format that frameworks and drivers like web or databases use. The entire Model View Controller (MVC) architecture is contained here. Sometimes they are also called the Gateways.

4) **Frameworks and Drivers:** This layer interacts with the external systems by implementing the interfaces defined in the previous layer. One interface can be implemented in several ways since the system might demand that. This can include web framework or database.

A. Communication between layers

Communication between layers in clean architecture is considered the Golden Rule of clean architecture. Since deeper layers mean more abstraction and outer layers mean real implementation, two simple rules are to be followed while layers communicate with each other. [1]

- **Simple structures:** When the outer layers communicate with the inner ones, they use structures defined by the inner layers to pass data across.

- **Interfaces:** When inner layers communicate with the outer layers, the interfaces are used. An interface is created when an outer layer plugs itself into the inner layer at the time of creation.

B. Characteristics of Clean Architecture

Some of the common characteristic's clean architecture shares with the other DDD architectures are the following:

- **Independent of Frameworks:** The system uses the framework to achieve functionality and is not confined by its constraints.

- **Testable:** The business rules can be tested independent of the finer details like databases, web or interfaces.

- **Independent of UI:** The UI can be changed without the implementation changing.

- **Independent of Database:** The database used can be changed without affecting the business logic or rules.

- **Independent of External Agents:** The business logic cannot be affected by any external factors or agents.

C. SOLID Principles

Guidelines on how to organise the functions and data into classes are provided by SOLID principles. Additionally, it describes the lines of communication between these classes. These principles are not exclusive to Object Oriented Programming, despite what class implies. These guidelines aid in the development of adaptable, simple to comprehend, and reusable components. [5]

1) **Single Responsibility Principle (SRP):** Every module or class in the system only has one task to complete according to the SRP principle.

2) **Open-Closed Principle (OCP):** OCP means that the system must be allowed to change through addition of new code but not change of existing code.

3) **Liskov Substitution Principle (LSP):** According to LSP, systems must be constructed with interchangeable components that can be swapped out for one another.

4) **Interface Segregation Principle (ISP):** According to ISP, a component cannot depend on another component if it is not using it.

5) **Dependency Inversion Principle (DIP):** According to DIP, the execution of lower-level implementations cannot affect the application of high-level policy. In other words, the policies must dictate the implementation.

V. PROPOSED APPROACH

The system under discussion was the financial system's existing MVP. The goal of this paper was to highlight the transformation of this MVP into a microservice made of several services, each of which adhered to the clean architecture principles.

The system has been broken into five services, namely financialGateway service, financialOffloader service, financialComputation service, financialValidator service and financial DataRetrieve service. These services each adopt the

clean architecture approach. The transformation of this system involves structuring the services into five layers. The five parts are Application, Gateway, Presentation, Shared and Framework. A package of modules named the library package containing the code frequently used by other services is also built in addition to these services.

Continuous integration and deployment (CICD) are used in the creation of the services. Despite the tech stack flexibility provided by the MSA, Python was used exclusively for the development since its built-in libraries made coding simple and efficient. A Python GraphQL API named Graphene was utilised to facilitate communication between the services. Either web based GraphQL APIs or tools like Postman can be used to access the interfaces.

VI. IMPLEMENTATION

The transformation of the monolithic system into a microservice started with identifying the main business logics in the system. After careful consideration the system was decomposed into five independent services that communicate with one another through defined GraphQL interfaces. Also, a common package was defined other than the services for the reusable code.

The decomposition of the monolithic system was done as follows. The five services of the microservice are financialGateway service, financialOffloader service, financialComputation service, financialValidator service and financialDataRetrieve service. The system and its five services are shown in Fig. 2. An overview of these services is given below:

- financialGateway service: As the name suggests this service authorizes and authenticates the users' requests that enter the microservice. Functions are provided according to the Service Level Agreements (SLA).
- financialOffloader service: This service provides accepts requests and offloads details about the requests.
- financialPricer service: All the data manipulation and core business purpose are achieved here.
- financialData service: As the name suggests this service helps in fetching data from various sources and databases.
- financialValidator service: Through this service the computations done by the system is validated frequently.

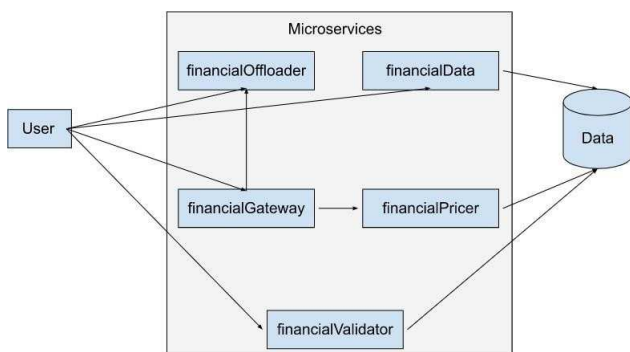


Fig. 2. Architecture Diagram of the microservice

All the services follow the same structure of CA, five layers that have distinguished purpose. This architecture is shown in Fig. 3. The services follow the four layered CA as mentioned below:

1) Entities:

- Domain Logic: Each service's business purpose is enclosed here. This means the main purpose of the service, similar to the description about the services.

2) Use Cases:

- Shared: Specific use case scenarios of the microservice are enclosed here. This includes details about the clients and the SLA they have agreed upon. The end points which the service accesses are here too. Additionally, reusable code in the service is placed here.

3) Interface Adapters:

- Gateway: Movement of data from and to each service is handled here. For communication GraphQL interfaces are utilized. Authentication and authorization are also performed here.

- Presentation: Here the data manipulation and main function calls are carried out here.

4) Frameworks and Drivers:

- Application: Here the interfaces that a user is exposed to are included. It includes the web and servers.
- Framework: Here the supporting tasks or automated jobs are included. It could include event triggers or event listeners.

Library package contains modules that are common to all services. This module can be extended by any service though if something were to change here then the services would not be affected.

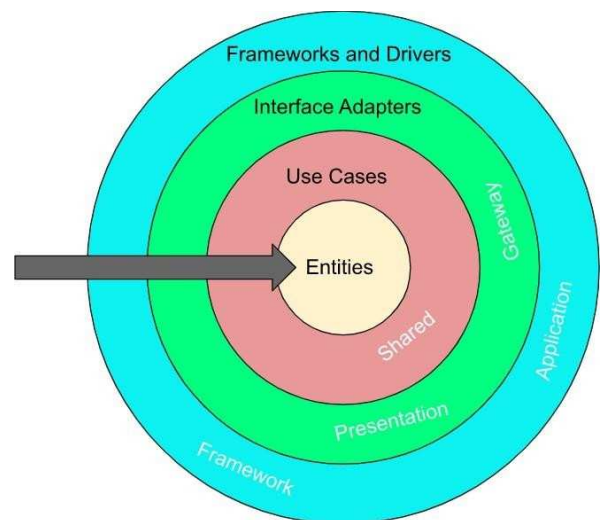


Fig. 3. Clean Architecture in each Service

VII. RESULTS AND DISCUSSION

The performance of the microservice was undoubtedly enhanced and made faster than a monolithic architecture using clean architecture in the services. Due to concurrent development, the development process was quick. When additional features needed to be developed, the effective architecture made the process simpler.

The code base is more loosely connected since the services adhered to the SOLID principles of the clean architecture. As a result, the code's usability increased. Owing to the easy

extensibility of individual services, scalability can be achieved as a whole or where required.

Thanks to SoC, testing was streamlined and handling of errors was simple. Testing of different levels was made easier by the Python unit test module's feature, mock patches. To test each layer independently, mock objects were used instead of other layer connections.

It is now simpler to add new features to the code that already exists without changing the legacy service. Clean architecture has also made the code easier to read. Even project comprehension, project navigation, and change of ownership of the system are now easier.

ACKNOWLEDGMENT

I truly thank everyone who helped me with this project and paper. Dr. Vishalakshi Prabhu H and my advisors deserve a special mention. I want to express my gratitude to R V College of Engineering, its staff members and professors for giving me a chance to work on this project. Last but not least, I'd like to acknowledge and express my gratitude to my parents and friends for their steadfast support during this process.

REFERENCES

- [1] Uncle Bob, The Clean Coder Blog: The Clean Architecture, Accessed on May 14, 2023 [Online], Available: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- [2] Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design (Robert C. Martin Series), 1st Edition, Pearson Education, Inc., 2018, ISBN-13: 978-0-13-449416-6
- [3] Leonardo Giordani, Clean Architectures in Python: A Practical Approach to Better Software Design, 2nd Edition, The Digital Cat Books, 2022, Available: <https://freecomputerbooks.com/Clean-Architectures-in-Python.html>
- [4] Sara Miteva, "The Concept of Domain-Driven Design Explained", Accessed on May 14, 2023 [Online], Available: <https://medium.com/microtica/the-concept-of-domain-driven-design-explained-3184c0fd7c3f>.
- [5] Felipe De Freitas, "Architecture inDDD", Accessed on May 14, 2023 [Online], Available: <https://thedomaindrivendesign.io/architecture-in-ddd/>
- [6] Zhiding Li, Chenqi Shang, Jianjie Wu, Yuan Li, "Microservice extraction based on knowledge graph from monolithic applications, Information and Software Technology", Volume 150, 2022, ISSN 0950-5849, doi:10.1016/j.infsof.2022.106992.
- [7] T. Salah, M. Jamal Zemerly, Chan Yeob Yeun, M. Al-Qutayri and Y. Al-Hammadi, "The evolution of distributed systems towards microservices architecture," 2016 11th International Conference for Internet Technology and Secured Transactions (ICITST), Barcelona, Spain, 2016, pp. 318-325, doi: 10.1109/ICITST.2016.7856721.
- [8] Muhammad Waseem, Peng Liang, Mojtaba Shahin, "A Systematic Mapping Study on Microservices Architecture in DevOps, Journal of Systems and Software", Volume 170, 2020, ISSN 0164-1212, doi:10.1016/j.jss.2020.110798.
- [9] L. Sun, Y. Li and R. A. Memon, "An open IoT framework based on microservices architecture," in China Communications, vol. 14, no. 2, pp. 154-162, February 2017, doi: 10.1109/CC.2017.7868163.
- [10] Shanshan Li, He Zhang, Zijia Jia, Chenxing Zhong, Cheng Zhang, Zhihao Shan, Jinfeng Shen, Muhammad Ali Babar, "Understanding and addressing quality attributes of microservices architecture: A Systematic literature review, Information and Software Technology", Volume 131, 2021, ISSN 0950-5849, doi:10.1016/j.infsof.2020.106449.
- [11] N. Alshuqayran, N. Ali and R. Evans, "A Systematic Mapping Study in Microservice Architecture," 2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA), Macau, China, 2016, pp. 44-51, doi: 10.1109/SOCA.2016.15.
- [12] Abdelhakim Hannousse, Salima Yahiouche, "Securing microservices and microservice architectures: A systematic mapping study, Computer Science Review", Volume 41, 2021, ISSN 1574-0137, doi:10.1016/j.cosrev.2021.100415.
- [13] C. M. Aderaldo, N. C. Mendonça, C. Pahl and P. Jamshidi, "Benchmark Requirements for Microservices Architecture Research," 2017 IEEE/ACM 1st International Workshop on Establishing the Community-Wide Infrastructure for Architecture-Based Software Engineering (ECASE), Buenos Aires, Argentina, 2017, pp. 8-13, doi: 10.1109/ECASE.2017.4.
- [14] Y. N. Nugroho, D. S. Kusumo and M. J. Alibasa, "Clean Architecture Implementation Impacts on Maintainability Aspect for Backend System Code Base," 2022 10th International Conference on Information and Communication Technology (ICoICT), Bandung, Indonesia, 2022, pp. 134-139, doi: 10.1109/ICoICT55009.2022.9914890.
- [15] Arango, E.C., Loaiza, O.L., "SCRUM Framework Extended with Clean Architecture Practices for Software Maintainability", Volume 230, Springer Cham, doi:10.1007/978-3-030-77442-4-56
- [16] Brown, P., Dimitrova, V., Hart, G., Cohn, A., and Moura, P. (2021). Refactoring the Whitby Intelligent Tutoring System for Clean Architecture. Theory and Practice of Logic Programming, 21(6), 818-834. doi:10.1017/S1471068421000326
- [17] M. Bukovcan, D. Blaz'evic', K. Nenadic' and M. Stevic', "Clean Architecture of Client-Side Software Development for Smart Furniture Control," 2022 11th Mediterranean Conference on Embedded Computing (MECO), Budva, Montenegro, 2022, pp. 1-4, doi: 10.1109/MECO55406.2022.9797122.
- [18] Aguilar, Pablo, and L. Figueira. "Clean Architecture is not only about business logic.", I Workshop de Tecnologia da Fatec Ribeirão Preto, Volume 1, 2020.
- [19] Sanchez, Daniel, Alix E. Rojas, and Hector Florez. "Towards a Clean Architecture for Android Apps using Model Transformations", IAENG International Journal of Computer Science 49.1, 2022, 270-278.
- [20] S. Boukhary and E. Colmenares, "A Clean Approach to Flutter Development through the Flutter Clean Architecture Package," 2019 International Conference on Computational Science and Computational Intelligence (CSCI), Las Vegas, NV, USA, 2019, pp. 1115-1120, doi: 10.1109/CSCI49370.2019.00211.
- [21] Bui, Duy. "Reactive programming and clean architecture in Android development", 2017.